Evolução e arquitetura do chatbot agronômico

Implementação com RAG LangChain LangGraph e API de fazenda

Relatório técnico • 13 de outubro de 2026 • Versão 1

Objetivo e conclusão

O projeto evoluiu de um protótipo de consulta a documentos para uma aplicação local que combina recuperação de conhecimento agronômico, medições meteorológicas simuladas e um plano fictício de plantio. A integração está funcional para demonstração e testes. Ainda não constitui um sistema validado de recomendação agronômica ou autorização de plantio.

Este documento explica a implementação existente, a função de cada tecnologia, o processamento de tokens, os resultados observados e o trabalho necessário para transformar o protótipo em um serviço confiável. O público principal é a equipe de desenvolvimento e os profissionais que irão revisar os critérios agronômicos.

O que existe e o que ainda é proposta

Situação	Componentes
Implementado	Interface de terminal Python; RAG documental; dois grafos LangGraph; modelos locais no Ollama; API JSON Server; plano e validações de dados simulados.
Verificado em testes	Percursos do grafo, tratamento de falhas, associação entre cadastro e fazenda, comparação numérica e integração HTTP. Há exemplos com Ollama e Chroma reais.
Não implementado	Aplicativo web ou móvel, backend público do chatbot, autenticação, memória persistente, sensores reais, previsão meteorológica e integração oficial de zoneamento.
Ainda não demonstrado	Precisão agronômica em amostra representativa, confiabilidade de decisão em campo e desempenho sob múltiplos usuários.

Como ler este relatório

As seções 1 a 5 descrevem evolução, tecnologias, arquitetura e dados. As seções 6 a 8 explicam tokens, exemplos e validação. As seções 9 a 11 apresentam os dados reais, a arquitetura futura e os critérios de aceitação. A seção 12 reúne referências e comandos.

LangChain e LangGraph são os componentes utilizados. Não há blockchain nesta arquitetura. O armazenamento da base vetorial também não modifica os pesos do modelo: RAG e treinamento são processos diferentes.

1 Evolução do projeto

Preparação do protótipo recebido

A etapa inicial examinou os arquivos recebidos, as dependências Python e a execução local. Foram tratados problemas de ambiente e de abertura do índice vetorial. Nesta instalação, o banco ativo foi mantido em um caminho externo sem acentos; a cópia original permaneceu como backup. Isso é uma característica observada neste ambiente, não uma exigência universal do Chroma.

A organização separou entradas de execução, implementação em src, configuração, documentos, scripts, estudos e testes. O ambiente utilizado passou a ser .venv312. A pasta do projeto posteriormente recebeu o nome chatbot.

Orquestração com LangGraph

O fluxo linear de buscar e responder recebeu avaliação de evidências, reformulação limitada e saída de esclarecimento. A pergunta original permanece no estado durante uma consulta. Há no máximo duas buscas documentais. Falhas de avaliação estruturada e falhas de infraestrutura são distinguidas da ausência de informação.

Otimização de desempenho

O diagnóstico identificou que a avaliação dos trechos consumia a maior parte da latência. A saída do avaliador foi encurtada, eliminando IDs duplicados e justificativas extensas. A janela de contexto passou de 8.192 para 4.096 tokens, e o modelo foi configurado para permanecer carregado por até 15 minutos. Uma regressão na seleção de fontes foi detectada durante os testes e corrigida antes de aceitar a mudança.

Integração da fazenda simulada

O JSON Server passou a disponibilizar medições e, depois, um plano de plantio com solo, cultivar e janela fictícia. O grafo externo escolhe documentos, API ou ambos. Consultas simples à API dispensam geração por LLM. Para uma faixa explícita de temperatura de semeadura, o Python realiza uma comparação verificável com os valores obtidos.

Correções de consistência e operação

- A umidade volumétrica do solo passou a ter uma única origem no plano do talhão; o campo duplicado das medições foi removido.
- Planos ausentes, inválidos, antigos ou com restrições bloqueiam a análise combinada de plantio antes da geração.
- A API passou a recarregar o JSON alterado e a rejeitar conteúdo inválido, sem reutilizar silenciosamente dados antigos.
- A mudança do nome da pasta revelou vínculos absolutos quebrados nas dependências Node. A instalação foi refeita com nodeLinker hoisted.
- Mensagens de erro foram ajustadas para distinguir servidor indisponível, recurso HTTP ausente e porta já ocupada.

2 Tecnologias utilizadas

As versões abaixo foram verificadas no ambiente Python local para este relatório. Elas não substituem um arquivo de dependências reproduzível: requirements.txt ainda contém faixas abertas em várias bibliotecas.

Tecnologia	Versão ou modelo	Responsabilidade no projeto
Python	3.12 no ambiente .venv312	Executa o chatbot, valida dados e coordena chamadas.
LangChain Core	1.6.10	Mensagens, Document, prompts e composição de operações.
LangGraph	1.2.14	Nós, estado, transições condicionais e streaming de eventos.
LangChain Ollama	1.1.0	Integra o Python aos modelos locais.
Ollama e Llama	llama3.2:3b	Serviço de inferência e modelo gerador/avaliador; 3b indica aproximadamente três bilhões de parâmetros.
Embeddings	nomic-embed-text	Representa texto em vetores para recuperação semântica.
Chroma e integração	1.5.9 e langchain-chroma 1.1.0	Persiste vetores, textos e metadados; recupera trechos por similaridade.
Pydantic	2.14.1	Valida formatos, tipos, limites e estruturas recebidas.
pypdf e Rich	6.20.0 e 15.0.0	Extração textual de PDFs e interface de terminal.
Node e JSON Server	24.19.0 e 0.17.4	Executam a API REST fictícia sobre api/db.json.
pnpm e unittest	Gerenciador e biblioteca padrão	Instala dependências Node e executa testes Python.

LangChain conecta as operações; LangGraph controla a ordem e as condições de execução. O LLM produz linguagem e classificações, mas não é o banco de dados. Chroma responde à busca documental; JSON Server expõe dados estruturados. A documentação oficial distingue essas responsabilidades [R1–R4].

No protótipo, não estão em uso LangSmith, treinamento supervisionado do modelo, fine tuning, aplicação móvel, serviços de nuvem ou blockchain. Recursos disponíveis nas bibliotecas não devem ser confundidos com recursos efetivamente configurados.

3 Arquitetura e fluxo de execução

A aplicação atual é uma CLI. Ela não possui uma API própria de chat: chama diretamente AgriculturalRAG no processo Python. O HTTP existente atende os dados da fazenda, enquanto o Ollama atende inferência local. O banco vetorial é acessado pela integração Chroma.

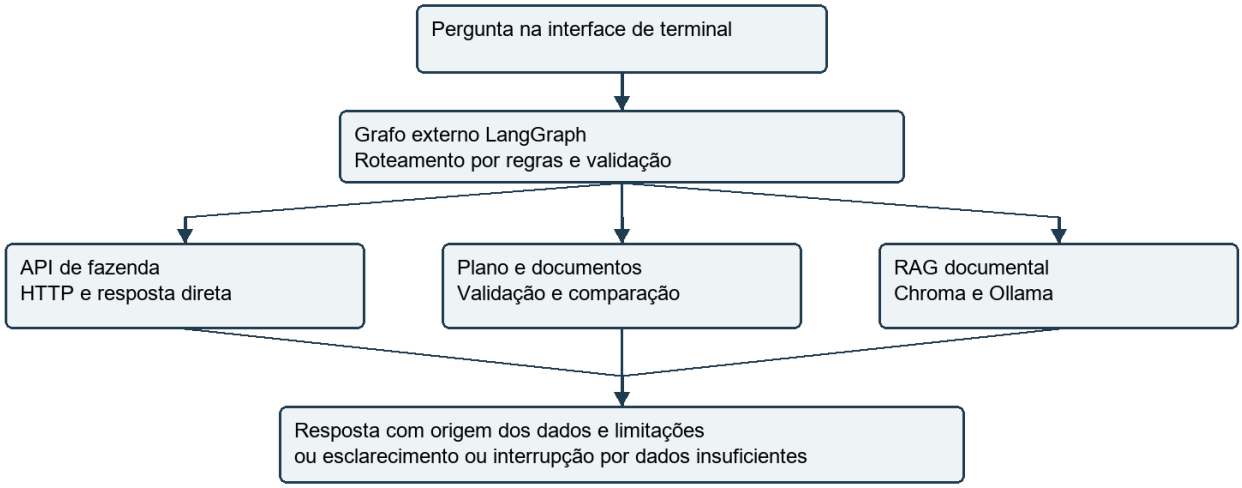


Figura 1. Fluxo atual. A comparação determinística e a geração por modelo são caminhos distintos, escolhidos conforme a operação e o formato da evidência.

Duas camadas de grafo

assistant_graph.py roteia entre responder_api, documentos e combinar_fontes. O roteamento usa regras de texto em português; não é um classificador de intenção universal. A consulta simples a sensores ou ao plano pode terminar sem chamar o modelo.

rag_graph.py implementa buscar, avaliar, reformular, responder, esclarecer e sem_evidencia. O avaliador retorna decisão estruturada e passagens. A aplicação valida a atribuição das passagens antes de aprovar fontes. Se a primeira busca for insuficiente, há uma reformulação; a segunda tentativa encerra o ciclo.

Estado e memória

O estado guarda pergunta original, consulta de busca, filtro, número de tentativas, documentos, avaliação e resultado. API e plano são obtidos para a consulta em curso. Não há histórico persistente, checkpoint ou memória entre mensagens; “e amanhã?” não recebe automaticamente o contexto da pergunta anterior.

4 Preparação da base e recuperação documental

Ingestão existente

src/ingest.py seleciona arquivos pelo DOC_METADATA_MAP, extrai texto por página com pypdf e trata relatórios Markdown. O mapa contém seis PDFs e dois relatórios técnicos. Existe outro PDF no acervo que não está nesse mapa; presença na pasta não comprova que um documento foi indexado.

O texto é limpo e dividido com RecursiveCharacterTextSplitter: chunk_size=750 e chunk_overlap=120. Nessa configuração, os tamanhos são medidos em caracteres, não em tokens. O embedding de cada trecho é calculado pelo nomic-embed-text e armazenado na coleção embrapa_conhecimento junto ao texto e aos metadados.

Dado persistido	Uso
Texto do trecho	Conteúdo fornecido ao avaliador e ao gerador.
Vetor de embedding	Busca por proximidade semântica; não é uma probabilidade de verdade.
source e titulo	Identificação do arquivo e nome da publicação.
cultura	Filtro de recuperação por domínio/cultura.
pagina e total_paginas	Localização no arquivo; pagina usa o índice do PDF mais um, que pode divergir da paginação impressa.

Recuperação durante uma pergunta

A pergunta é convertida em embedding; Chroma retorna os quatro candidatos configurados por k=4, respeitando o filtro quando ativo. O avaliador verifica se os trechos sustentam uma resposta. Até duas passagens podem compor a seleção compacta. Comparar por similaridade semântica aproxima assuntos; não garante que a pergunta foi respondida ou que o trecho contém todas as condições necessárias [R2, R4].

Limitações que ainda precisam ser corrigidas

A retomada da ingestão usa a contagem já existente para cortar a lista de trechos. Isso não detecta corretamente arquivos alterados, removidos ou reordenados. Além disso, o código pode exibir conclusão após falha persistente de um lote. O teste de diagnóstico existente reproduz essas falhas; ele não significa que elas foram resolvidas.

Antes de atualizar o corpus em produção, são necessários IDs estáveis por documento/versão/trecho, hashes do conteúdo, atualização idempotente, remoção de versões antigas e confirmação dos lotes realmente gravados. Deve-se preservar a estrutura das perguntas e respostas e recuperar passagens vizinhas quando uma condição atravessar uma página.

5 API e contratos de dados

O JSON Server expõe REST em 127.0.0.1:3001. O cliente Python usa GET com timeout de três segundos por requisição e valida os dados. Não há cache meteorológico no cliente. O servidor recarrega alterações externas do JSON; conteúdo inválido produz HTTP 503. Esse comportamento foi testado com um banco temporário.

Recurso	Campos principais	Papel
/fazendas	id, nome, município, UF, fuso	Identifica a unidade consultada.
/medicoes	fazendaId, talhaoId, observado_em, temperaturas, chuva, vento, umidade do ar	Última observação meteorológica simulada.
/planos_plantio	cultura, cultivar, ciclo, solo, horário, janela e regras de teste	Cadastro do talhão e condição hídrica simulada.
/health	service, version, simulado	Identifica o processo da API; não comprova validade agronômica.

Uma origem para cada informação

A umidade do solo fica somente em planos_plantio.umidade_volumetrica_pct. O antigo campo umidade_solo_pct das medições foi removido para impedir respostas contraditórias. Umidade volumétrica de 28% é uma observação distinta dos 68% de umidade relativa do ar. Chuva acumulada de 12 mm também não é uma medição da umidade do solo.

Os vínculos entre fazenda, talhão, cultura, município e UF são conferidos antes de aceitar o plano. Mais de um plano aplicável ou medições de talhões diferentes não são resolvidos por escolha arbitrária. A demonstração ainda não possui uma interface completa de seleção de talhão.

Atualidade e regras artificiais

A observação deve ser do mesmo dia em Cuiabá, sem horário futuro e com idade de até seis horas. Esse limite é uma política do protótipo; não foi validado como frequência agronômica adequada. A observação do solo possui horário próprio. Datas de início e fim da janela são inclusivas.

A cultivar SOJA-DEMO-001, seu ciclo de 115 dias, a janela de 01/10 a 30/11/2026 e o intervalo hídrico de 20–35% foram inventados para testar o software. A fonte da janela é SIMULACAO_NAO_ZARC. Nenhuma dessas informações representa validação oficial da cultivar ou consulta ao zoneamento.

Executar seed.cjs substitui o JSON pelos valores padrão e atualiza horários. Isso cria novos dados fictícios; não coleta sensores. Alterar JSON ou executar seed não retreina o LLM e não requer nova ingestão no Chroma.

6 Tokens e custo de processamento

O que é um token

Token é uma unidade de representação do texto usada pelo modelo. Pode corresponder a uma palavra, parte dela, pontuação ou combinação de caracteres. O tokenizador transforma essas unidades em IDs. Não existe equivalência fixa entre palavras, caracteres e tokens; a divisão depende do vocabulário do modelo e do idioma [R5].

Por exemplo, “Qual é a temperatura do solo?” precisa ser tokenizado pelo modelo utilizado para obter a contagem real. Separar a frase por espaços não fornece essa contagem. O Llama e o modelo de embeddings podem empregar tokenizadores diferentes. Um token também não é uma coordenada do vetor de embedding.

Entrada saída e contexto

A entrada inclui instruções, pergunta, esquema estruturado, documentos e dados da API quando enviados ao modelo. A saída contém o texto ou JSON gerado. Para um modelo causal, o orçamento de contexto precisa acomodar a entrada e a continuação gerada. A configuração atual usa num_ctx=4096; esse número não significa 4.096 palavras nem 4.096 tokens disponíveis somente para a resposta.

Parâmetro atual	Interpretação
num_ctx = 4096	Janela configurada por chamada. É necessário contabilizar prompts, dados e espaço para saída.
num_predict = 400	Limite de saída do avaliador/reformulador. Não é a contagem efetivamente consumida; atingir o limite pode truncar JSON.
temperature = 0 ou 0,2	Controle de amostragem do modelo. Não corresponde a graus Celsius nem garante correção factual.
keep_alive = 15m	Retenção solicitada do modelo carregado; não é memória da conversa.

Exemplo de orçamento apenas ilustrativo

Exemplo didático: 4.096 − 3.000 tokens de entrada − 400 de saída = 696 de margem. A entrada deve incluir os marcadores de chat. O protótipo ainda não gerencia esse orçamento automaticamente; esses valores não são medições do projeto.

Por que várias etapas ficam mais lentas

Uma resposta documental comum pode usar uma chamada de avaliação e outra de geração. A reformulação adiciona uma chamada e uma nova avaliação. Cada chamada reprocessa sua própria entrada. Já respostas meteorológicas diretas e a comparação numérica implementada não geram tokens de saída pelo LLM, embora a comparação possa ter passado antes pelo avaliador documental.

Ollama expõe prompt_eval_count, eval_count e durações para medir entrada, saída e tempo. Tokens por segundo podem ser calculados como eval_count dividido por eval_duration em segundos. O uso local não cria cobrança de tokens por um provedor externo, mas consome CPU/GPU, memória, energia e tempo. A aplicação ainda precisa registrar essas métricas por etapa [R6].

7 Exemplo completo de análise de plantio

Entrada: “Quais as condições climáticas hoje? Posso plantar soja?”. O exemplo abaixo corresponde aos dados simulados do projeto e demonstra o encadeamento da informação, não uma orientação para plantio real.

Origem	Dados relevantes	Interpretação implementada
API meteorológica	Ar 28 °C; solo 25 °C; chuva 12 mm; vento 8 km/h	Apresenta os valores com unidade, horário e origem [API].
Plano do talhão	Umidade volumétrica 28%; sem encharcamento; trafegável	Verifica preenchimento e atualidade; compara a umidade com a regra fictícia.
Calendário de teste	01/10 a 30/11/2026; consulta em 13/10/2026	Data dentro da janela fictícia; isso não comprova ZARC aplicável.
Documento recuperado	Faixa de 20 a 30 °C para semeadura da soja	Extrai limites do trecho aprovado e faz comparação numérica.

Como a execução escolhe a resposta

O grafo consulta a API e valida a medição. Depois consulta o plano da soja e verifica seus vínculos e restrições. Se houver pendências ou condições desfavoráveis nas regras fictícias, encerra a análise combinada com os motivos, sem usar a temperatura para ignorá-los.

Se o plano passar, a busca documental procura condições de temperatura para semeadura. Quando encontra uma frase no formato reconhecido, o código extrai os limites numéricos; não usa 20 e 30 como constantes agronômicas fixas. A comparação informa se as medições estão numericamente dentro ou fora do intervalo.

O ponto que exige cuidado

O trecho recuperado informa a faixa, mas não identifica explicitamente ar ou solo. O programa assinala essa ambiguidade. A página seguinte ou uma passagem complementar pode esclarecer o requisito, mas a recuperação atual não garante essa complementação. Também não valida se a cultivar fictícia realmente é apropriada ao local.

Uma conclusão fiel ao estágio atual é: “O cenário passou nas regras simuladas de solo e calendário. As temperaturas estão numericamente no intervalo recuperado, mas a interpretação desse requisito precisa ser confirmada. Não há evidência suficiente para transformar o resultado em recomendação real de plantio.”

Outras perguntas

“Qual a umidade do solo hoje?” consulta o plano único aplicável e mostra a umidade uma vez. “Qual a temperatura ideal para semear soja?” segue o fluxo documental. “Vai chover amanhã?” não pode ser respondida com a última medição: a API atual não fornece previsão.

8 Verificação qualidade e desempenho

O que foi testado

A última revisão funcional registrou 47 testes Python e quatro testes com JSON Server real. Após reparar a instalação na pasta renomeada, os quatro testes HTTP foram executados novamente. Os testes cobrem decisões do grafo, limite de buscas, fontes inválidas, isolamento de consultas, validade temporal, vínculo do plano, condições desfavoráveis e persistência/recarga do JSON.

Há também registros de execução com Chroma e Ollama reais. Eles mostram que as integrações funcionam em exemplos selecionados. Aprovar testes de software ou percorrer o nó correto não equivale a medir precisão agronômica; um teste anterior seguiu o caminho esperado e, mesmo assim, gerou uma resposta factual incorreta.

Medição local registrada	Tempo	Condição
Origem da soja antes da otimização	41,57 s	Diagnóstico com avaliação de 34,09 s.
Origem da soja após otimização	5,58 s	Modelos já carregados; não é comparação controlada.
Umidade do solo após revisão	0,22 s	API e plano, sem geração pelo LLM.
Plantio após revisão da integração	5,62 s	API, plano e recuperação documental.

Os números são observações de execuções distintas, não um benchmark ou promessa de latência. Carregamento inicial, cache, concorrência, tamanho dos textos e divisão CPU/GPU alteram os resultados. Em diagnóstico anterior, o contexto menor aumentou a parcela reportada de GPU de 68% para 78%; isso não é utilização instantânea medida da GPU.

Limites do avaliador e das citações

A avaliação usa o mesmo modelo pequeno que gera respostas. Pydantic verifica estrutura, não a veracidade da conclusão. A conferência de passagens tolera espaços, acentos, caixa e hifenização; ela confirma correspondência textual normalizada, mas não demonstra relevância semântica. Em testes, houve omissão de marcadores de citação e seleção de fonte desnecessária.

Métricas necessárias para uma avaliação real

- Recuperação: proporção de perguntas cujo trecho esperado aparece entre os candidatos, separada por cultura e operação.
- Resposta: sustentação das afirmações, correção das citações, unidades e identificação do talhão e da data.
- Decisão: falsos favoráveis, falsas restrições, abstenção adequada e discordância com revisão agronômica.
- Operação: tempo até o primeiro conteúdo útil, latência p50/p95, erros HTTP e consumo de tokens por etapa.

9 Dados necessários para uma decisão confiável

Um chatbot que apoia decisões precisa de dados corretos, critérios aplicáveis e avaliação independente. Acrescentar campos fictícios ajuda a exercitar o código, mas não substitui coleta, calibração ou validação técnica. Os requisitos abaixo são propostas de evolução, não recursos já disponíveis.

Grupo de dados	Conteúdo necessário	Controle de qualidade
Identificação espacial	Fazenda, talhão, coordenadas, área, município e histórico de uso	Vincular sensores e recomendações ao talhão correto; evitar médias sem representatividade.
Solo e água	Textura, profundidade, umidade volumétrica, método, capacidade de campo, drenagem e condição operacional	Calibração, unidade e profundidade de leitura; limiares definidos por especialista para o contexto.
Meteorologia	Séries de chuva, temperatura, vento e previsão quando necessária	Origem, horário de coleta/emissão, horizonte, resolução espacial, lacunas e incerteza.
Cultura e cultivar	Identificação real, grupo/ciclo, sementes e características pertinentes	Cadastro verificável, adequação regional e validade das informações.
Calendário aplicável	Regras oficiais vigentes e suas condições de aplicação	Compatibilizar cultura, município, solo, ciclo/grupo, safra e versão da fonte.
Manejo e operação	Sistema de cultivo, resíduos, irrigação, condição do terreno e objetivo da operação	Critérios revisados por responsável técnico; não inferir apenas de uma medição meteorológica.

Zoneamento e atualidade das fontes

O ZARC é um instrumento de gestão de risco climático. Sua aplicação exige consultar os resultados e atos pertinentes ao caso; uma janela genérica de calendário não o substitui. A futura integração deve registrar origem, safra/versão e parâmetros utilizados, em vez de tratar o nome do município como condição suficiente [R7].

Conhecimento documental adequado

O corpus deve separar manuais agronômicos de relatórios técnicos sobre IA, registrar edição, data, cultura, região e condições de validade. Trechos que dependem de tabelas, figuras ou páginas vizinhas precisam manter essas relações. Regras aplicáveis ao solo não podem ser substituídas por medições do ar apenas porque os números são semelhantes.

A política de dados ausentes deve ser explícita: perguntar ao usuário, buscar outra fonte permitida ou abster-se. “Não foi encontrado” não significa zero, ausência de risco ou inexistência da informação em todo o acervo.

10 Arquitetura proposta para um aplicativo

A próxima aplicação pode conservar o núcleo Python e os grafos, mas deve separar interface, serviço de chat e serviços de dados. Esta seção descreve uma proposta; esses componentes não foram implementados no protótipo atual.

Interface e backend de chat

Uma interface web ou móvel permitiria selecionar fazenda, talhão, cultura e operação, apresentar a última atualização dos dados e receber a resposta com fontes. Um backend Python, por exemplo com FastAPI, exporia POST /chat e uma transmissão de eventos por SSE. A interface não deve acessar Chroma ou executar modelos diretamente.

O backend autenticaria o usuário, verificaria autorização para o talhão, atribuiria um identificador à consulta e executaria o LangGraph. O estado da conversa poderia ser persistido por sessão; memória deve guardar identidade e proveniência sem reutilizar automaticamente medições vencidas. Mensagens de progresso não precisam expor detalhes internos irrelevantes ao usuário.

Serviços de dados e persistência

JSON Server seria substituído por APIs reais de estações, sensores, cadastro agrícola e fontes oficiais. Um banco transacional poderia manter fazendas, talhões, permissões e auditoria; séries temporais armazenariam medições. Chroma continuaria como índice documental, com versões e rotina de atualização. Cada armazenamento possui uma finalidade diferente.

Separar regras e linguagem

A camada de decisão deve executar critérios agronômicos explícitos e versionados, aprovados por especialistas. O modelo pode interpretar a pergunta, apoiar a recuperação e explicar o resultado. Ele não deve transformar lacunas em fatos nem criar um limiar operacional sem fonte. Regras conflitantes ou evidências insuficientes devem produzir esclarecimento ou encaminhamento para revisão humana.

Segurança e observabilidade

- Autenticação e autorização por fazenda/talhão, TLS, gestão de segredos e proibição de acesso cruzado entre usuários.
- Permissão de leitura para o chatbot; escritas em cadastro ou comandos sobre equipamentos exigiriam fluxos próprios e autorização.
- Validação de URLs e ferramentas permitidas; documentos e respostas de API tratados como dados, não instruções.
- Logs com versão do modelo, regras, fontes, horários e motivos da decisão; proteção de informações pessoais e comerciais.
- Timeouts, controle de concorrência, limites de consumo, recuperação de falhas e verificação de dados vencidos.

A API fictícia atual permite operações de alteração de dados e não possui autenticação. Está vinculada ao loopback para testes. Ela não deve ser publicada na internet como backend de produção.

11 Plano de evolução e critérios de aceitação

Etapa 1 Consolidar a engenharia da informação

Corrigir a atualização incremental da ingestão, fixar dependências Python, versionar o corpus e registrar métricas por chamada. Acrescentar recuperação de passagens adjacentes, tratamento de tabelas e seleção por assunto. O critério de aceitação é demonstrar que um documento alterado atualiza corretamente suas evidências e que falha de lote não termina como sucesso.

Etapa 2 Validar o domínio agronômico

Definir com um profissional agronômico quais decisões o produto poderá apoiar. Transformar cada decisão em requisitos de dados, regras, exceções e fontes. Criar uma base de perguntas e casos com resultado esperado, incluindo situações desfavoráveis, limites, dados conflitantes e ausência de dados. Separar casos de desenvolvimento dos casos reservados para avaliação.

Um resultado “favorável” deve exigir evidências aplicáveis e atuais para todos os critérios obrigatórios do escopo. Se um critério não puder ser avaliado, a saída será “dados insuficientes” ou “revisão necessária”. A aprovação final deve incluir revisão agronômica; um número de testes verdes não substitui essa revisão.

Etapa 3 Conectar dados reais e construir a aplicação

Integrar uma fazenda piloto e dados medidos com origem e unidade verificáveis. Substituir a janela fictícia por consulta aplicável e versionada. Implementar backend de chat, identidade do usuário, seleção de talhão, histórico auditável e interface com conclusão primeiro, evidências depois e pendências explícitas.

Etapa 4 Realizar um piloto supervisionado

Comparar o sistema com avaliações independentes, registrar divergências e revisar regras. Medir latência em carga controlada, além de erros de decisão. Ampliar culturas e operações somente após validação de cada escopo; o sucesso do exemplo de soja não valida automaticamente irrigação, pulverização, outras regiões ou outros cultivos.

Critério de liberação	Evidência esperada
Dados e proveniência	Nenhuma conclusão usa silenciosamente talhão errado, informação vencida ou fonte sem identificação.
Regras	Condições operacionais justificadas e aprovadas, sem limiares fictícios na execução real.
Respostas	Afirmações sustentadas, citações verificáveis e abstenção correta nos casos definidos como críticos.
Operação	Metas de latência e disponibilidade acordadas e medidas; falhas recuperáveis e trilha de auditoria.

A equipe técnica e o responsável agronômico devem definir as metas antes do piloto. Ainda não há evidência para declarar uma taxa de acerto ou confiabilidade de campo.

12 Referências e reprodução

Fontes técnicas externas

[R1] LangGraph overview. Abrir referência

[R2] Retrieval na documentação LangChain. Abrir referência

[R3] JSON Server versão 0.17.4. Abrir referência

[R4] Introdução ao Chroma. Abrir referência

[R5] Tokenization algorithms da Hugging Face. Abrir referência

[R6] Ollama API chat e métricas de uso. Abrir referência

[R7] Ministério da Agricultura e Pecuária e ZARC. Abrir referência

Referências externas consultadas em 13 de outubro de 2026. As descrições da implementação se referem ao código e aos registros locais do projeto, não a todos os recursos possíveis das bibliotecas.

Evidências locais e configuração

Código principal: src/assistant_graph.py, src/rag_graph.py, src/rag_engine.py, src/weather_api.py, src/planting_context.py e src/ingest.py. API: api/server.cjs, api/seed.cjs, api/db.json e api/pnpm-workspace.yaml. Caminho do banco: config/chroma_path.txt, com precedência de RAG_CHROMA_DIR.

Registros: tests/resultados_langgraph_antes_otimizacao.json, tests/resultados_langgraph_aquecido.json e tests/resultados_correcao_integracao.json. Os registros são amostras históricas; as medições fictícias atuais podem ter sido renovadas pelo seed.

Comandos na raiz da pasta chatbot

```
.\iniciar_api.bat
.\iniciar_chatbot.bat
.\parar_api.bat
.\renovar_dados_demo.bat
.\.venv312\Scripts\python.exe -m unittest discover -s tests -p "test_*.py"
.\.venv312\Scripts\python.exe -m tests.verificar_api_http
```

A renovação da simulação substitui os valores editados pelos padrões do seed. Os quatro testes HTTP usam banco temporário; os testes unitários não exigem uma estação meteorológica. Para testes reais do chatbot, Ollama, base vetorial e API precisam estar disponíveis.